

Flask in Python – Complete Detailed Guide

1. Introduction to Flask

What is Flask?

Flask is a lightweight, open-source Python web framework used to develop web applications, REST APIs, dashboards, and microservices.

Flask is called a **Micro Framework** because it provides only the essential components required for web development and allows developers to add extensions as needed.

Definition

Flask is a minimalistic Python web framework that enables developers to build web applications quickly while providing complete control over application architecture and components.

Why Flask?

Traditional Web Development:

HTML
CSS
JavaScript
Backend Logic
Database Connectivity
Routing
Authentication

Requires extensive setup.

With Flask:

Python Code
↓
Web Application

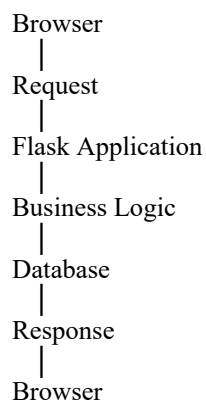
A simple web application can be created with just a few lines of code.

Features of Flask

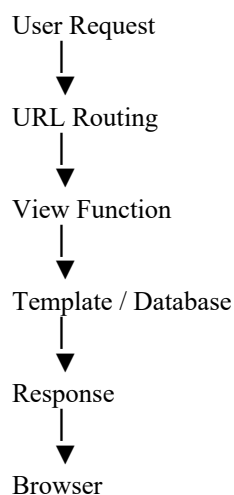
- Lightweight Framework
- Easy to Learn
- Flexible Architecture
- Built-in Development Server
- URL Routing
- Template Engine (Jinja2)
- REST API Support
- Database Integration
- Session Management
- Extension-Based Design

Flask Architecture

Flask follows a simple architecture:



Flask Request-Response Cycle



Installing Flask

Install Using PIP

```
pip install flask
```

Check Installation:

```
pip show flask
```

First Flask Application

Create:

```
app.py
```

Code:

```
from flask import Flask

app = Flask(__name__)

@app.route("/")
def home():
    return "Welcome to Flask"

if __name__ == "__main__":
    app.run(debug=True)
```

Run:

```
python app.py
```

Output:

```
http://127.0.0.1:5000
```

Browser:

```
Welcome to Flask
```

Flask Application Object

```
app = Flask(__name__)
```

Purpose

Creates a Flask application instance.

Example:

```
from flask import Flask  
  
app = Flask(__name__)
```

URL Routing

Routing maps URLs to functions.

```
@app.route("/")  
def home():  
    return "Home Page"
```

Multiple Routes

```
@app.route("/about")  
def about():  
    return "About Page"  
  
@app.route("/contact")  
def contact():  
    return "Contact Page"
```

URLs:

```
http://localhost:5000/about  
http://localhost:5000/contact
```

Dynamic Routing

```
@app.route("/student/<name>")
def student(name):
    return f'Welcome {name}'
```

Example:

```
/student/John
```

Output:

```
Welcome John
```

HTTP Methods

Flask supports:

- GET
- POST
- PUT
- DELETE
- PATCH

Example:

```
@app.route("/login",
            methods=["GET","POST"])
def login():
    pass
```

Request Object

Used to access client data.

```
from flask import request
```

Example:

```
@app.route("/login",
            methods=["POST"])
def login():

    username = request.form["username"]

    return username
```

Response Object

Flask automatically returns responses.

```
return "Hello"
```

Custom response:

```
from flask import Response

return Response(
    "Success",
    status=200
)
```

Templates in Flask

Flask uses the Jinja2 template engine.

Folder Structure:

```
project
├── app.py
├── templates
│   └── home.html
```

HTML Template

home.html

```
<!DOCTYPE html>

<html>

<body>

<h1>Welcome</h1>

</body>

</html>
```

Rendering Templates

```
from flask import render_template

@app.route("/")
def home():

    return render_template(
        "home.html"
    )
```

Template Variables

Python:

```
@app.route("/")
def home():

    return render_template(
        "home.html",
        name="John"
    )
```

HTML:

```
<h1>{{ name }}</h1>
```

Output:

```
John
```

Template Loops

```
{% for student in students %}

<p>{{ student }}</p>

{% endfor %}
```

Template Conditions

```
{% if marks > 40 %}
```

Pass

```
{% else %}
```

Fail

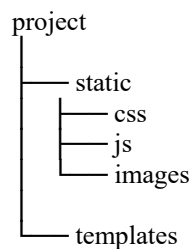
```
{% endif %}
```

Static Files

Used for:

- CSS
- JavaScript
- Images

Structure:



CSS Example

```
<link rel="stylesheet"
      href="{{ url_for(
        'static',
        filename='css/style.css'
      ) }}">
```

Forms in Flask

HTML Form:

```
<form method="POST">

<input type="text"
```

```
        name="username">

<input type="submit">

</form>
```

Form Handling

```
from flask import request

@app.route("/form",
          methods=["GET", "POST"])

def form():

    if request.method == "POST":

        name = request.form["username"]

        return name

    return render_template(
        "form.html"
    )
```

Redirects

```
from flask import redirect

return redirect("/home")
```

URL Generation

```
from flask import url_for

url_for("home")
```

Output:

```
/
```

Sessions

Store user-specific information.

```
from flask import session  
  
app.secret_key = "secret"
```

Store:

```
session["user"] = "John"
```

Retrieve:

```
session["user"]
```

Cookies

Create cookie:

```
response.set_cookie(  
    "username",  
    "John"  
)
```

Read cookie:

```
request.cookies.get(  
    "username"  
)
```

Database Connectivity

Flask supports:

- MariaDB
- MySQL
- PostgreSQL
- SQLite
- MongoDB

SQLite Example

```
import sqlite3

conn = sqlite3.connect(
    "students.db"
)
```

Flask + MariaDB Example

```
import mariadb

conn = mariadb.connect(
    host="localhost",
    user="root",
    password="password",
    database="studentdb"
)
```

Flask-SQLAlchemy

Install:

```
pip install flask-sqlalchemy
```

ORM Example

```
from flask_sqlalchemy import SQLAlchemy

db = SQLAlchemy(app)
```

Model:

```
class Student(db.Model):

    id = db.Column(
        db.Integer,
        primary_key=True
    )
```

```
name = db.Column(
    db.String(100)
)
```

CRUD Operations

Insert

```
student = Student(
    name="John"
)

db.session.add(student)

db.session.commit()
```

Read

```
Student.query.all()
```

Update

```
student = Student.query.get(1)

student.name = "David"

db.session.commit()
```

Delete

```
db.session.delete(student)

db.session.commit()
```

Flask Blueprints

Blueprints organize large applications.

Application

- Student Blueprint
- Faculty Blueprint
- Admin Blueprint

Example:

```
from flask import Blueprint

student_bp = Blueprint(
    "student",
    __name__
)
```

Error Handling

```
@app.errorhandler(404)

def page_not_found(error):

    return "Not Found", 404
```

REST APIs with Flask

Install:

```
pip install flask-restful
```

Example:

```
from flask import jsonify

@app.route("/api/student")

def student():

    return jsonify({

        "name": "John",

        "marks": 85

    })
```

Output:

```
{  
  "name": "John",  
  "marks": 85  
}
```

Authentication

Install:

```
pip install flask-login
```

Provides:

- Login
- Logout
- User Sessions
- Access Control

Flask Project Structure

```
project  
├── app.py  
├── config.py  
├── models.py  
├── forms.py  
├── routes.py  
├── static  
│   ├── css  
│   ├── js  
│   └── images  
├── templates  
│   ├── home.html  
│   ├── login.html  
│   └── dashboard.html  
└── database.db
```

Flask and Bootstrap

Responsive UI:

```
<link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">
```

Benefits:

- Mobile Friendly
- Professional Design
- Responsive Layout

Flask Application Architecture

```
Browser
|
Bootstrap UI
|
Flask Routes
|
Business Logic
|
Database Layer
|
MariaDB/MySQL
```

Flask Deployment

Deploy using:

- Amazon Web Services
- Microsoft Azure
- Google Cloud
- Linux VPS
- Docker Containers

Flask + Docker

Dockerfile:

```
FROM python:3.11

WORKDIR /app

COPY . .
```

```
RUN pip install -r requirements.txt
```

```
CMD ["python","app.py"]
```

Build:

```
docker build -t flask-app .
```

Run:

```
docker run -p 5000:5000 flask-app
```

Flask vs Django

Feature	Flask	Django
Framework Type	Micro	Full Stack
Learning Curve	Easy	Moderate
ORM	Optional	Built-in
Admin Panel	No	Built-in
Flexibility	Very High	Moderate
Enterprise Features	Extension-Based	Built-in

Flask vs Streamlit

Feature	Flask	Streamlit
Web Applications	Excellent	Limited
Dashboards	Good	Excellent
APIs	Excellent	Limited
HTML Templates	Yes	No
Authentication	Extension-Based	Basic
Enterprise Apps	Excellent	Moderate

Real-World Applications

Flask is commonly used for:

- Student Management Systems
- ERP Systems
- E-Commerce Websites
- REST APIs
- Machine Learning Portals
- Cyber Security Dashboards

- IoT Applications
- AI Web Interfaces

Advantages of Flask

1. Lightweight and simple.
2. Easy to learn.
3. Highly flexible.
4. Extension ecosystem.
5. REST API friendly.
6. Suitable for microservices.
7. Excellent for custom architectures.

Limitations of Flask

1. No built-in admin panel.
2. Requires additional extensions.
3. More setup needed for large applications.
4. Less structured than Django.

Summary

Flask is a lightweight, flexible Python web framework designed for rapid web development and API creation. It follows a minimalistic approach, providing essential features such as routing, request handling, template rendering, session management, and database connectivity while allowing developers to choose their own architecture and extensions. Flask is widely used for web applications, REST APIs, machine learning interfaces, microservices, cyber security dashboards, and cloud-native applications. Due to its simplicity and flexibility, Flask is often the preferred framework for beginners learning web development as well as professionals building customized solutions.

Experiment 01: Step by step process to build a program to print "Hello World".

Flask Program: Print "Hello World" (Step-by-Step Process)

This is the first program every beginner should create while learning **Flask**. The application creates a small web server that displays **"Hello World"** in a web browser.

Step 1: Install Python

First, verify that Python is installed.

Open Terminal/Command Prompt:

```
python --version
```

or

```
python3 --version
```

Example Output:

```
Python 3.13.2
```

If Python is not installed, download it from:

[Python Official Website](https://www.python.org/)

Step 2: Create a Project Folder

Create a new folder:

```
FlaskHelloWorld
```

Navigate into the folder:

```
mkdir FlaskHelloWorld  
cd FlaskHelloWorld
```

Step 3: Create a Virtual Environment (Recommended)

Create virtual environment:

```
python -m venv venv
```

Activate Virtual Environment

Windows

```
venv\Scripts\activate
```

Linux/Mac

```
source venv/bin/activate
```

Successful activation:

```
(venv) C:\FlaskHelloWorld>
```

Step 4: Install Flask

Install Flask using pip:

```
pip install flask
```

Verify installation:

```
pip show flask
```

Example Output:

```
Name: Flask  
Version: 3.x.x
```

Step 5: Create Python File

Create a file named:

```
app.py
```

Project Structure:

```
FlaskHelloWorld  
├── venv  
└── app.py
```

Step 6: Write Flask Code

Open app.py and write:

```
from flask import Flask

app = Flask(__name__)

@app.route("/")
def home():
    return "Hello World"

if __name__ == "__main__":
    app.run(debug=True)
```

Step 7: Understanding the Code

Import Flask

```
from flask import Flask
```

Imports the Flask class from the Flask package.

Create Application Object

```
app = Flask(__name__)
```

Creates a Flask application instance.

`__name__` tells Flask where the application is located.

Define Route

```
@app.route("/")
```

This is called a **Route Decorator**.

It maps the URL:

```
http://localhost:5000/
```

to a Python function.

Create View Function

```
def home():  
    return "Hello World"
```

When the user visits the URL, Flask executes this function and returns:

```
Hello World
```

Start Flask Server

```
if __name__ == "__main__":  
    app.run(debug=True)
```

This starts the Flask development server.

debug=True

Provides:

- Automatic restart
- Error messages
- Debugging support

Step 8: Run the Application

Open terminal:

```
python app.py
```

Output:

```
* Serving Flask app 'app'  
* Debug mode: on  
  
Running on:  
http://127.0.0.1:5000
```

Step 9: Open Browser

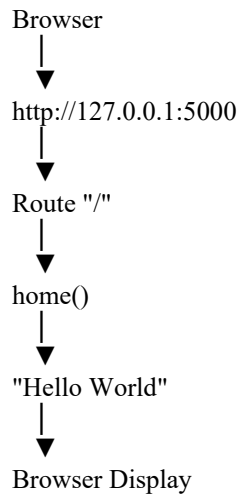
Open:

```
http://127.0.0.1:5000
```

Output:

Hello World

Step 10: Understanding Request Flow



Enhanced Version with HTML

Instead of plain text:

```
from flask import Flask

app = Flask(__name__)

@app.route("/")
def home():

    return """
    <html>
      <body>
        <h1>Hello World</h1>
      </body>
    </html>
    """

if __name__ == "__main__":
    app.run(debug=True)
```

Output in browser:

Hello World

displayed as a heading.

Version Using HTML Template

Create Structure

```
FlaskHelloWorld
├── app.py
└── templates
    └── home.html
```

home.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Flask Demo</title>
</head>
<body>

<h1>Hello World</h1>

</body>
</html>
```

app.py

```
from flask import Flask
from flask import render_template

app = Flask(__name__)

@app.route("/")
def home():

    return render_template(
        "home.html"
    )

if __name__ == "__main__":
    app.run(debug=True)
```

Output

Open:

```
http://127.0.0.1:5000
```

Browser displays:

```
Hello World
```

Common Errors

ModuleNotFoundError

```
ModuleNotFoundError:  
No module named 'flask'
```

Solution:

```
pip install flask
```

Port Already in Use

Address already in use

Solution:

```
app.run(  
    debug=True,  
    port=5001  
)
```

Wrong File Name

Make sure the file is:

```
app.py
```

and run:

```
python app.py
```

Summary

The Flask "Hello World" application demonstrates the fundamental concepts of Flask:

1. Install Flask.
2. Create a Flask application object.
3. Define a route using `@app.route("/")`.
4. Create a view function that returns content.
5. Start the development server using `app.run()`.
6. Access the application through a browser at:

```
http://127.0.0.1:5000
```

This simple application forms the foundation for building larger Flask applications such as Student Management Systems, E-Commerce Websites, REST APIs, Machine Learning Portals, and Enterprise Web Applications.

Experiment 02: Write a program to accept the marks of 5 subjects (english, hindi, maths, sst, science) from a student, calculate their total and percentage and store it in the CSV file using pandas. Moreover add themes to the program.

a complete Flask + Pandas + Bootstrap Theme application that:

Features

- Accepts Student Name
- Accepts marks of:
 - o English
 - o Hindi
 - o Maths
 - o SST
 - o Science
- Calculates:
 - o Total
 - o Percentage
- Stores data in CSV using Pandas
- Displays result on screen
- Uses a professional Bootstrap theme
- Responsive web interface

Project Structure

```
StudentResultSystem
├── app.py
├── student_marks.csv
├── templates
│   ├── index.html
│   └── result.html
├── static
│   └── css
```

Step 1: Install Required Libraries

```
pip install flask pandas
```

Step 2: Flask Application (app.py)

```
from flask import Flask
from flask import render_template
from flask import request

import pandas as pd
import os

app = Flask(__name__)

@app.route("/")
def home():

    return render_template(
        "index.html"
    )

@app.route(
    "/calculate",
    methods=["POST"]
)
def calculate():

    student_name = request.form["student_name"]

    english = int(request.form["english"])
    hindi = int(request.form["hindi"])
    maths = int(request.form["maths"])
    sst = int(request.form["sst"])
    science = int(request.form["science"])

    total = (
        english +
        hindi +
        maths +
        sst +
        science
    )

    percentage = total / 5

    data = {

        "Student Name": [student_name],
        "English": [english],
        "Hindi": [hindi],
        "Maths": [maths],
        "SST": [sst],
```

```

        "Science":[science],
        "Total":[total],
        "Percentage":[percentage]

    }

    df = pd.DataFrame(data)

    filename = "student_marks.csv"

    if os.path.exists(filename):

        df.to_csv(
            filename,
            mode="a",
            header=False,
            index=False
        )

    else:

        df.to_csv(
            filename,
            index=False
        )

    return render_template(
        "result.html",
        student_name=student_name,
        total=total,
        percentage=round(
            percentage,
            2
        )
    )

if __name__ == "__main__":
    app.run(
        debug=True
    )

```

Step 3: Student Input Page (templates/index.html)

Using Bootstrap Darkly Theme

```

<!DOCTYPE html>
<html>

<head>

```

```
<title>
Student Result System
</title>

<link
rel="stylesheet"
href="https://cdn.jsdelivr.net/npm/bootswatch@5.3.3/dist/darkly/bootstrap.min.css">

</head>

<body>

<div class="container mt-5">

<div class="card">

<div class="card-header">

<h2>
Student Result System
</h2>

</div>

<div class="card-body">

<form
action="/calculate"
method="POST">

<div class="mb-3">

<label>
Student Name
</label>

<input
type="text"
name="student_name"
class="form-control"
required>

</div>

<div class="row">

<div class="col-md-6 mb-3">

<label>
English
</label>
```

```
<input
type="number"
name="english"
class="form-control"
required>

</div>

<div class="col-md-6 mb-3">

<label>
Hindi
</label>

<input
type="number"
name="hindi"
class="form-control"
required>

</div>

<div class="col-md-6 mb-3">

<label>
Maths
</label>

<input
type="number"
name="maths"
class="form-control"
required>

</div>

<div class="col-md-6 mb-3">

<label>
SST
</label>

<input
type="number"
name="sst"
class="form-control"
required>

</div>
```

```
<div class="col-md-12 mb-3">
```

```
<label>  
Science  
</label>
```

```
<input  
type="number"  
name="science"  
class="form-control"  
required>
```

```
</div>
```

```
</div>
```

```
<button  
class="btn btn-success">
```

```
Calculate Result
```

```
</button>
```

```
</form>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
</body>
```

```
</html>
```

Step 4: Result Page (templates/result.html)

```
<!DOCTYPE html>  
<html>
```

```
<head>
```

```
<title>  
Result  
</title>
```

```
<link  
rel="stylesheet"  
href="https://cdn.jsdelivr.net/npm/bootswatch@5.3.3/dist/darkly/bootstrap.min.css">
```

```
</head>

<body>

<div class="container mt-5">

<div class="card">

<div class="card-header">

<h2>
Result
</h2>

</div>

<div class="card-body">

<h3>

Student:
{{ student_name }}

</h3>

<h4>

Total Marks:
{{ total }}

</h4>

<h4>

Percentage:
{{ percentage }} %

</h4>

<a
href="/"
class="btn btn-primary">

Add Another Student

</a>

</div>

</div>
```

```
</div>

</body>

</html>
```

CSV Output

After submitting records:

```
Student Name,English,Hindi,Maths,SST,Science>Total,Percentage
John,85,78,90,88,92,433,86.6
Alice,75,80,82,79,88,404,80.8
```

Popular Bootstrap Themes

You can change the theme by replacing the CSS URL.

Darkly (Dark Theme)

```
https://cdn.jsdelivr.net/npm/bootswatch@5.3.3/dist/darkly/bootstrap.min.css
```

Cosmo

```
https://cdn.jsdelivr.net/npm/bootswatch@5.3.3/dist/cosmo/bootstrap.min.css
```

Flatly

```
https://cdn.jsdelivr.net/npm/bootswatch@5.3.3/dist/flatly/bootstrap.min.css
```

Lux

```
https://cdn.jsdelivr.net/npm/bootswatch@5.3.3/dist/lux/bootstrap.min.css
```

Minty

```
https://cdn.jsdelivr.net/npm/bootswatch@5.3.3/dist/minty/bootstrap.min.css
```

Running the Application

```
python app.py
```

Open:

```
http://127.0.0.1:5000
```

Learning Concepts Covered

- Flask Application Development
- Routing (@app.route)
- HTML Forms
- POST Method
- Request Handling
- Bootstrap Themes
- Responsive Web Design
- Pandas DataFrame
- CSV File Handling
- Result Calculation
- Template Rendering using Jinja2
- Dynamic Web Pages

This project is an excellent beginner-to-intermediate Flask practical that combines **Web Development, Data Processing, Pandas, File Handling, and Responsive UI Design.**

Experiment 03: Store the student data in both CSV and MariaDB.

Step 1: Install Required Packages

```
pip install flask pandas mariadb
```

Step 2: Create MariaDB Database

Login to MariaDB:

```
mariadb -u root -p
```

Create database and table:

```
CREATE DATABASE studentdb;

USE studentdb;

CREATE TABLE student_marks
(
    id INT AUTO_INCREMENT PRIMARY KEY,
    student_name VARCHAR(100),
    english INT,
    hindi INT,
    maths INT,
```

```
sst INT,  
science INT,  
total INT,  
percentage DECIMAL(5,2)  
);
```

Verify:

```
DESC student_marks;
```

Step 3: Project Structure

```
StudentResultSystem  
├── app.py  
├── student_marks.csv  
├── templates  
│   ├── index.html  
│   └── result.html
```

Step 4: Update Flask Application (app.py)

```
from flask import Flask  
from flask import render_template  
from flask import request  
  
import pandas as pd  
import mariadb  
import os  
  
app = Flask(__name__)  
  
# -----  
# Database Connection  
# -----  
  
def get_connection():  
  
    conn = mariadb.connect(  
        host="localhost",  
        port=3306,  
        user="root",  
        password="your_password",  
        database="studentdb"  
    )  
  
    return conn
```

```

# -----
# Home Page
# -----

@app.route("/")
def home():

    return render_template(
        "index.html"
    )

# -----
# Calculate Result
# -----

@app.route(
    "/calculate",
    methods=["POST"]
)

def calculate():

    student_name = request.form["student_name"]

    english = int(
        request.form["english"]
    )

    hindi = int(
        request.form["hindi"]
    )

    maths = int(
        request.form["maths"]
    )

    sst = int(
        request.form["sst"]
    )

    science = int(
        request.form["science"]
    )

    # Calculate Result

    total = (
        english +
        hindi +

```

```

    maths +
    sst +
    science
)

percentage = total / 5

# -----
# Save to CSV
# -----

data = {

    "Student Name":
        [student_name],

    "English":
        [english],

    "Hindi":
        [hindi],

    "Maths":
        [maths],

    "SST":
        [sst],

    "Science":
        [science],

    "Total":
        [total],

    "Percentage":
        [percentage]

}

df = pd.DataFrame(data)

filename = "student_marks.csv"

if os.path.exists(filename):

    df.to_csv(
        filename,
        mode="a",
        header=False,
        index=False
    )

```

```

else:

    df.to_csv(
        filename,
        index=False
    )

# -----
# Save to MariaDB
# -----

try:

    conn = get_connection()

    cursor = conn.cursor()

    query = """
    INSERT INTO student_marks
    (
        student_name,
        english,
        hindi,
        maths,
        sst,
        science,
        total,
        percentage
    )
    VALUES
    (
        ?, ?, ?, ?, ?, ?, ?, ?
    )
    """

    cursor.execute(

        query,

        (
            student_name,
            english,
            hindi,
            maths,
            sst,
            science,
            total,
            percentage
        )
    )

```

```

    )

    conn.commit()

    cursor.close()
    conn.close()

except mariadb.Error as e:

    return f"Database Error: {e}"

return render_template(
    "result.html",
    student_name=student_name,
    total=total,
    percentage=round(
        percentage,
        2
    )
)

# -----
# View All Students
# -----

@app.route("/students")

def students():

    conn = get_connection()

    query = """
    SELECT *
    FROM student_marks
    """

    df = pd.read_sql(
        query,
        conn
    )

    conn.close()

    return df.to_html(
        classes="table table-striped"
    )

# -----

```

```
if __name__ == "__main__":

    app.run(
        debug=True
    )
```

Step 5: Add a Button in result.html

Update result.html:

```
<a href="/"
class="btn btn-primary">
Add Another Student
</a>

<a href="/students"
class="btn btn-success">
View All Students
</a>
```

Step 6: Sample Output in MariaDB

```
SELECT * FROM student_marks;
```

id	student_name	english	hindi	maths	sst	science	total	percentage
1	John	85	78	90	88	92	433	86.60
2	Alice	75	80	82	79	88	404	80.80

Step 7: View Data Through Browser

Open:

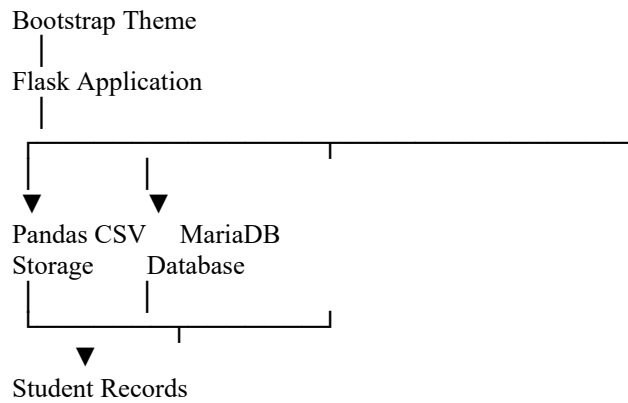
```
http://127.0.0.1:5000/students
```

Output:

```
+-----+
| Student Records          |
+-----+
| John  85 78 90 88 92 433 86.60 |
| Alice 75 80 82 79 88 404 80.80 |
+-----+
```

Improved Professional Architecture

Browser
|



Concepts Covered

Flask

- Routing
- Forms
- POST Requests
- Templates
- Jinja2

Pandas

- DataFrame Creation
- CSV Storage
- CSV Append Operations

MariaDB

- Database Connectivity
- SQL INSERT
- SQL SELECT
- Transactions
- Cursor Operations

Frontend

- Bootstrap Theme
- Responsive Forms
- Responsive Tables